

C# TOÀN TẬP

PHẦN 8.6

GENERIC METHOD

Type Parameter - Type Inference - Return T - Nhiều kiểu generic

Mục tiêu của bài học

Hiểu generic method và type parameter T.
Biết nhận parameter kiểu T, trả về T và dùng nhiều type parameter.
Phân biệt generic method với overload và object.
Nhận biết giới hạn của T và chuẩn bị cho generic constraints.
Hoàn thành bài tập GenericHelper trước khi sang Generic Class.

Tài liệu học tập - Không kèm đáp án bài tập

Phần 8.6 - Generic Method trong C#

Bạn đã sử dụng generic rất nhiều:

```
List<Student>
Dictionary<string, Product>
HashSet<string>
Queue<Order>
Stack<string>
```

Trong các ví dụ trên, Student, string, Product và Order là những kiểu dữ liệu được truyền vào generic.

Bây giờ, thay vì chỉ sử dụng class generic có sẵn của .NET, chúng ta sẽ tự tạo generic method.

1. Generic method là gì?

Generic method là method có thể làm việc với nhiều kiểu dữ liệu khác nhau mà không cần viết lại nhiều lần.

Ví dụ bình thường:

```
public static void DisplayInt(int value)
{
    Console.WriteLine(value);
}

public static void DisplayString(string value)
{
    Console.WriteLine(value);
}

public static void DisplayDouble(double value)
{
    Console.WriteLine(value);
}
```

Ba method trên có logic giống nhau, chỉ khác kiểu dữ liệu. Ta có thể thay bằng một generic method:

```
public static void Display<T>(T value)
{
    Console.WriteLine(value);
}
```

Sử dụng:

```
Display<int>(10);
Display<string>("Hello");
Display<double>(8.5);
```

Hoặc viết ngắn hơn:

```
Display(10);
Display("Hello");
Display(8.5);
```

C# sẽ tự suy luận kiểu của T.

2. Cú pháp generic method

Cú pháp tổng quát:

```
accessModifier returnType MethodName<T>(parameters)
```

```
{
}
```

Ví dụ:

```
public static void Display<T>(T value)
{
    Console.WriteLine(value);
}
```

Thành phần	Ý nghĩa
public static	Phạm vi truy cập và loại method
void	Kiểu trả về
Display	Tên method
<T>	Khai báo kiểu generic
T value	Parameter có kiểu T

Dấu <T> sau tên method cho C# biết đây là generic method.

3. T là gì?

T là một type parameter, tức biến đại diện cho kiểu dữ liệu.

```
Display<int>(10); // T = int
Display<string>("C#"); // T = string
Display<Student>(student); // T = Student
```

T không phải một kiểu dữ liệu cụ thể. Nó chỉ là tên đại diện và sẽ được thay bằng kiểu thật khi method được gọi.

4. Có bắt buộc phải đặt tên là T không?

Không bắt buộc. Có thể viết:

```
public static void Display<TValue>(TValue value)
{
    Console.WriteLine(value);
}

public static void Display<TItem>(TItem item)
{
    Console.WriteLine(item);
}
```

Tên quy ước	Ý nghĩa thường dùng
T	Một kiểu chung
TItem	Kiểu phần tử
TKey	Kiểu khóa
TValue	Kiểu giá trị
TResult	Kiểu kết quả

Ví dụ quen thuộc:

```
Dictionary<TKey, TValue>
```

5. C# tự suy luận kiểu generic

Bạn có thể gọi đầy đủ:

```
Display<int>(10);
Display<string>("Hello");
```

Nhưng thông thường chỉ cần:

```
Display(10);
Display("Hello");
```

Type inference

10 -> int -> T là int

"Hello" -> string -> T là string

student -> Student -> T là Student

C# nhìn vào argument để suy luận kiểu. Đây gọi là type inference.

6. Generic method nhận nhiều parameter cùng kiểu

```
public static void DisplayPair<T>(
    T first,
    T second)
{
    Console.WriteLine($"First: {first}");
    Console.WriteLine($"Second: {second}");
}
```

Gọi với số nguyên:

```
DisplayPair(10, 20); // T = int
```

Gọi với chuỗi:

```
DisplayPair("C#", ".NET"); // T = string
```

Hai parameter đều có kiểu T, nên chúng phải tương thích với cùng một kiểu.

```
DisplayPair(10, "Hello");
```

Ví dụ trên có thể không suy luận được vì first là int, second là string và C# không xác định được một T duy nhất phù hợp với cả hai.

7. Generic method trả về T

Generic method không chỉ nhận T, nó còn có thể trả về T.

```
public static T GetFirst<T>(List<T> items)
{
    if (items == null)
    {
        throw new ArgumentNullException(
            nameof(items),
            "The list cannot be null.");
    }

    if (items.Count == 0)
    {
        throw new InvalidOperationException(
            "The list is empty.");
    }

    return items[0];
}
```

```
}
```

Dùng với int:

```
List<int> numbers = new List<int>
{
    10,
    20,
    30
};

int firstNumber = GetFirst(numbers); // T = int
```

Dùng với string:

```
List<string> names = new List<string>
{
    "An",
    "Binh",
    "Cuong"
};

string firstName = GetFirst(names); // T = string
```

Dùng với object tự tạo:

```
List<Student> students = new List<Student>
{
    student1,
    student2
};

Student firstStudent = GetFirst(students); // T = Student
```

Một method `GetFirst<T>()` có thể dùng cho mọi `List<T>`.

8. Generic method với mảng

Generic method không chỉ dùng với `List<T>`. Ví dụ lấy phần tử cuối của mảng:

```
public static T GetLast<T>(T[] items)
{
    if (items == null)
    {
        throw new ArgumentNullException(
            nameof(items),
            "The array cannot be null.");
    }

    if (items.Length == 0)
    {
        throw new InvalidOperationException(
            "The array is empty.");
    }

    return items[items.Length - 1];
}
```

Sử dụng với số:

```
int[] numbers =
{
    10,
    20,
    30
};
```

```
int lastNumber = GetLast(numbers); // 30
```

Sử dụng với chuỗi:

```
string[] names =
{
    "An",
    "Binh",
    "Cuong"
};

string lastName = GetLast(names); // Cuong
```

9. Method Swap<T>()

Một ví dụ phổ biến là đổi chỗ hai giá trị.

```
public static void Swap<T>(
    ref T first,
    ref T second)
{
    T temporary = first;
    first = second;
    second = temporary;
}
```

Sử dụng với int:

```
int number1 = 10;
int number2 = 20;

Swap(ref number1, ref number2);

Console.WriteLine(number1); // 20
Console.WriteLine(number2); // 10
```

Sử dụng với string:

```
string name1 = "An";
string name2 = "Binh";

Swap(ref name1, ref name2);

Console.WriteLine(name1); // Binh
Console.WriteLine(name2); // An
```

Vì sao cần ref?

Không có ref:

```
public static void Swap<T>(
    T first,
    T second)
```

Method chỉ đổi hai biến cục bộ bên trong method. Hai biến bên ngoài không thay đổi.

Có ref:

```
Swap(ref number1, ref number2);
```

Method được phép thay đổi trực tiếp hai biến bên ngoài. Phần trọng tâm hiện tại vẫn là generic; ref chỉ được dùng để minh họa.

10. Generic method có nhiều kiểu dữ liệu

Một method có thể khai báo nhiều type parameter:

```
public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)
{
    Console.WriteLine(
        $"Key: {key}, Value: {value}");
}
```

Lời gọi	TKey	TValue
DisplayKeyValue("SV01", "Nguyen Van An")	string	string
DisplayKeyValue("P01", 20_000_000m)	string	decimal
DisplayKeyValue(1, student)	int	Student

Đây cũng là cách Dictionary<TKey, TValue> hoạt động với hai kiểu generic.

11. Generic method trả về một kiểu khác

Generic method không bắt buộc phải trả về T. Ví dụ kiểm tra hai giá trị có bằng nhau không:

```
public static bool AreEqual<T>(
    T first,
    T second)
{
    return EqualityComparer<T>.Default.Equals(
        first,
        second);
}
```

```
bool result1 = AreEqual(10, 10);
bool result2 = AreEqual("C#", ".NET");
```

Thành phần	Kiểu
Parameter	T
Return	bool

Vì sao không viết trực tiếp first == second?

Với kiểu generic T, C# không chắc toán tử == có được hỗ trợ hay không. Do đó, cách tổng quát hơn là:

```
EqualityComparer<T>.Default.Equals(
    first,
    second);
```

Phần so sánh object chuyên sâu sẽ học sau. Hiện tại chỉ cần hiểu generic method không phải lúc nào cũng trả về T.

12. Generic method và method overload

Giả sử không dùng generic:

```
public static void Display(int value)
{
    Console.WriteLine(value);
}
```

```
public static void Display(string value)
{
    Console.WriteLine(value);
}

public static void Display(double value)
{
    Console.WriteLine(value);
}
```

Đây là method overload: cùng tên, khác kiểu parameter và mỗi kiểu có một method riêng.

Generic chỉ cần một method:

```
public static void Display<T>(T value)
{
    Console.WriteLine(value);
}
```

Trường hợp	Nên dùng
Logic giống nhau cho nhiều kiểu	Generic method
Mỗi kiểu cần xử lý khác nhau	Overload

Dùng generic khi logic giống nhau: hiển thị giá trị, đổi chỗ, lấy phần tử đầu/cuối, lưu trữ phần tử, kiểm tra bằng nhau.

Dùng overload khi mỗi kiểu cần xử lý khác nhau:

```
public static void Print(int value)
{
    Console.WriteLine($"Integer: {value}");
}

public static void Print(DateTime value)
{
    Console.WriteLine(
        value.ToString("dd/MM/yyyy"));
}
```

13. Generic khác object thế nào?

Trước generic, có thể viết:

```
public static void Display(object value)
{
    Console.WriteLine(value);
}
```

Method này cũng nhận được nhiều kiểu, nhưng object có hạn chế.

```
public static object GetFirst(
    List<object> items)
{
    return items[0];
}
```

Khi nhận kết quả:

```
object result = GetFirst(items);
Student student = (Student)result;
```

Muốn lấy lại Student phải ép kiểu. Nếu ép sai kiểu có thể xảy ra lỗi.

Với generic:

```
public static T GetFirst<T>(
    List<T> items)
{
    return items[0];
}

Student student = GetFirst(students);
```

Không cần ép kiểu.

Ưu điểm của generic

An toàn kiểu dữ liệu.
Không cần ép kiểu.
Phát hiện lỗi khi biên dịch.
Tái sử dụng code.
Dễ đọc hơn.

14. Giới hạn khi làm việc với T

Vì C# chưa biết T là kiểu gì, bạn không thể tùy ý dùng mọi phép toán.

```
public static T Add<T>(
    T first,
    T second)
{
    return first + second;
}
```

Đoạn trên thường không hợp lệ vì C# không biết T có hỗ trợ toán tử + hay không.

T có thể là int, string, Student hoặc Order. Không phải kiểu nào cũng cộng được.

Tương tự, không thể mặc định gọi:

```
value.DisplayInfo();
```

C# không biết kiểu T có method DisplayInfo() hay không.

Để yêu cầu T phải có một đặc điểm cụ thể, ta dùng generic constraint:

```
where T : ...
```

Phần này sẽ được học kỹ ở Phần 8.8.

15. Không thể luôn tạo new T()

Ví dụ sau chưa hợp lệ:

```
public static T Create<T>()
{
    return new T();
}
```

C# không biết T có constructor không tham số hay không.

Muốn làm vậy cần constraint:

```
public static T Create<T>()
    where T : new()
```

```
{
    return new T();
}
```

```
Student student = Create<Student>();
```

Student phải có constructor không tham số. where T : new() sẽ được học kỹ trong phần generic constraints.

16. Generic method có thể nằm ở đâu?

Generic method có thể nằm trong class bình thường:

```
public class GenericUtilities
{
    public static void Display<T>(T value)
    {
        Console.WriteLine(value);
    }
}
```

```
GenericUtilities.Display(10);
GenericUtilities.Display("Hello");
```

Nó cũng có thể là method instance:

```
public class ValuePrinter
{
    public void Display<T>(T value)
    {
        Console.WriteLine(value);
    }
}

ValuePrinter printer = new ValuePrinter();
printer.Display(10);
printer.Display("Hello");
```

Nếu method không phụ thuộc vào trạng thái của object, dùng static thường hợp lý.

17. Generic method trong class generic

Một class generic có thể chứa generic method:

```
public class Storage<T>
{
    private T? _value;

    public void Store(T value)
    {
        _value = value;
    }

    public void DisplayOther<TAnother>(
        TAnother value)
    {
        Console.WriteLine(value);
    }
}
```

Type parameter	Phạm vi
T	Thuộc về class Storage<T>
TAnother	Chỉ thuộc về method DisplayOther()

Phần class generic sẽ học ở bài tiếp theo.

18. Ứng dụng generic method trong .NET

Generic method xuất hiện rất nhiều trong .NET.

LINQ

```
Enumerable.First<T>();
Enumerable.Where<T>();
Enumerable.Select<TSource, TResult>();
```

JSON

```
JsonSerializer.Deserialize<Student>(json);
```

Trong đó T = Student.

Entity Framework Core

```
dbContext.Set<Product>();
dbContext.Set<Student>();
```

Dependency Injection

```
services.AddScoped<IProductService, ProductService>();
```

Generic giúp framework làm việc với nhiều kiểu mà vẫn an toàn kiểu dữ liệu.

19. Ví dụ hoàn chỉnh - GenericUtilities

```
public static class GenericUtilities
{
    public static void Display<T>(T value)
    {
        Console.WriteLine(value);
    }

    public static void DisplayPair<T>(
        T first,
        T second)
    {
        Console.WriteLine($"First: {first}");
        Console.WriteLine($"Second: {second}");
    }

    public static T GetFirst<T>(
        List<T> items)
    {
        ValidateList(items);
        return items[0];
    }

    public static T GetLast<T>(
        List<T> items)
    {
        ValidateList(items);
        return items[items.Count - 1];
    }

    public static void Swap<T>(
        ref T first,
        ref T second)
```

```

    {
        T temporary = first;
        first = second;
        second = temporary;
    }

    public static bool AreEqual<T>(
        T first,
        T second)
    {
        return EqualityComparer<T>
            .Default
            .Equals(first, second);
    }

    private static void ValidateList<T>(
        List<T> items)
    {
        if (items == null)
        {
            throw new ArgumentNullException(
                nameof(items),
                "The list cannot be null.");
        }

        if (items.Count == 0)
        {
            throw new InvalidOperationException(
                "The list is empty.");
        }
    }
}

```

Lưu ý: `ValidateList<T>()` cũng phải là generic vì nó nhận `List<T>`.

20. Sử dụng GenericUtilities

```

public class Program
{
    public static void Main(string[] args)
    {
        try
        {
            GenericUtilities.Display(10);
            GenericUtilities.Display("Hello C#");
            GenericUtilities.Display(8.5);

            List<int> numbers = new List<int>
            {
                10,
                20,
                30
            };

            int firstNumber =
                GenericUtilities.GetFirst(numbers);

            int lastNumber =
                GenericUtilities.GetLast(numbers);

            Console.WriteLine(
                $"First number: {firstNumber}");

            Console.WriteLine(
                $"Last number: {lastNumber}");

            List<string> names = new List<string>
            {
                "An",
                "Binh",
                "Cuong"
            };
        }
    }
}

```

```

        string firstName =
            GenericUtilities.GetFirst(names);

        string lastName =
            GenericUtilities.GetLast(names);

        Console.WriteLine(
            $"First name: {firstName}");

        Console.WriteLine(
            $"Last name: {lastName}");

        int number1 = 10;
        int number2 = 20;

        GenericUtilities.Swap(
            ref number1,
            ref number2);

        Console.WriteLine(
            $"Number 1: {number1}");

        Console.WriteLine(
            $"Number 2: {number2}");

        bool areEqual =
            GenericUtilities.AreEqual(
                "C#",
                "C#");

        Console.WriteLine(
            $"Are equal: {areEqual}");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine(
            $"Invalid argument: {ex.Message}");
    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine(
            $"Invalid operation: {ex.Message}");
    }
    catch (Exception ex)
    {
        Console.WriteLine(
            $"Unexpected error: {ex.Message}");
    }
}

```

21. Những lỗi thường gặp

Lỗi 1: Quên <T> sau tên method

Sai:

```

public static void Display(T value)
{
}

```

C# không biết T là gì. Đúng:

```

public static void Display<T>(T value)
{
}

```

Lỗi 2: Khai báo generic nhưng không dùng T

```
public static void Display<T>(  
    string value)  
{  
    Console.WriteLine(value);  
}
```

Method khai báo T nhưng không dùng đến T, nên generic không có ý nghĩa. Nên là:

```
public static void Display<T>(  
    T value)  
{  
    Console.WriteLine(value);  
}
```

Lỗi 3: Dùng toán tử không được bảo đảm

```
public static T Add<T>(  
    T first,  
    T second)  
{  
    return first + second;  
}
```

Không phải mọi kiểu T đều hỗ trợ phép cộng.

Lỗi 4: Gọi method riêng của một class trên T

```
public static void Display<T>(T value)  
{  
    value.DisplayInfo();  
}
```

C# không biết T có DisplayInfo() hay không. Cần constraint với interface hoặc class cha, sẽ học ở Phần 8.8.

Lỗi 5: Không validation collection

```
public static T GetFirst<T>(  
    List<T> items)  
{  
    return items[0];  
}
```

Có thể lỗi khi items == null hoặc items.Count == 0. Nên validation trước.

Lỗi 6: Nhầm T là object

T không phải luôn là object cụ thể. Nó có thể là int, double, bool, string, Student hoặc Product. Do đó không được giả định T luôn có property hoặc method nhất định.

22. Tóm tắt cần nhớ

Generic method

Method làm việc với nhiều kiểu dữ liệu.

Dùng type parameter như T.

C# có thể suy luận T từ argument.

Method có thể nhận T, trả về T hoặc trả về kiểu khác.

Một method có thể có nhiều type parameter như <TKey, TValue>.

Cú pháp:

```
public static void MethodName<T>(T value)
{
}
```

Method trả về T:

```
public static T GetValue<T>(T value)
{
    return value;
}
```

Nhiều kiểu generic:

```
public static void Display<TKey, TValue>(
    TKey key,
    TValue value)
{
}
```

Ưu điểm	Hạn chế hiện tại
Tái sử dụng code	Không tùy ý dùng +, -, >, <
An toàn kiểu	Không tùy ý gọi method riêng trên T
Không cần ép kiểu	Không tùy ý new T()
Giảm overload trùng logic	Cần constraint khi yêu cầu khả năng cụ thể
Phát hiện lỗi khi biên dịch	Không được giả định T có thành viên nào đó

Muốn giới hạn hoặc yêu cầu khả năng của T, cần generic constraints.

Bài tập 8.6 - Xây dựng GenericHelper

Tạo class static:

```
public static class GenericHelper
```

Method 1 - DisplayValue<T>()

```
public static void DisplayValue<T>(T value)
```

- Hiển thị kiểu dữ liệu của T.
- Hiển thị giá trị.

Gợi ý lấy kiểu:

```
typeof(T).Name
```

Kết quả ví dụ:

```
Type: Int32, Value: 10
Type: String, Value: Hello
```

Method 2 - GetFirstItem<T>()

```
public static T GetFirstItem<T>(
    List<T> items)
```

- items không được null.
- Danh sách không được rỗng.
- Trả về phần tử đầu tiên.

Method 3 - GetLastItem<T>()

```
public static T GetLastItem<T>(
    List<T> items)
```

- Validation giống method trên.
- Trả về phần tử cuối cùng.

Method 4 - SwapValues<T>()

```
public static void SwapValues<T>(
    ref T first,
    ref T second)
```

- Đổi chỗ hai giá trị.
- Dùng một biến tạm kiểu T.

Method 5 - AreValuesEqual<T>()

```
public static bool AreValuesEqual<T>(
    T first,
    T second)
```

Yêu cầu sử dụng:

```
EqualityComparer<T>.Default.Equals(
    first,
    second);
```

Method 6 - DisplayKeyValue<TKey, TValue>()

```
public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)
```

- Hiển thị key.
- Hiển thị value.
- Thử với nhiều cặp kiểu khác nhau.

Yêu cầu trong Program.cs

1. Gọi DisplayValue() với int, string, decimal và một object tự tạo.
2. Tạo List<int> và dùng GetFirstItem(), GetLastItem().
3. Tạo List<string> và dùng lại hai method trên.

4. Dùng SwapValues() với hai số nguyên.
5. Dùng SwapValues() với hai chuỗi.
6. Dùng AreValuesEqual() với hai số bằng nhau và hai chuỗi khác nhau.
7. Gọi DisplayKeyValue() với string-string, string-decimal và int-object.
8. Thử truyền một danh sách rỗng vào GetFirstItem().
9. Dùng try-catch để xử lý exception.

Ba khả năng trọng tâm cần luyện

Nhận parameter kiểu T.

Trả về kiểu T.

Dùng nhiều type parameter.

Checklist trước khi nộp bài

- Class được khai báo static đúng yêu cầu.
- Mỗi method khai báo <T> hoặc <TKey, TValue> đúng vị trí.
- Không lập code riêng cho int, string và object.
- GetFirstItem() và GetLastItem() kiểm tra null và danh sách rỗng.
- SwapValues() dùng ref và biến tạm kiểu T.
- AreValuesEqual() dùng EqualityComparer<T>.Default.Equals().
- Program.cs kiểm thử nhiều kiểu dữ liệu.
- Có test danh sách rỗng và xử lý exception.
- Không chép đáp án từ ví dụ; tự viết lại theo hiểu biết.

KẾT THÚC PHẦN 8.6 - GENERIC METHOD